

Reviewer Pack — Minimal Hyperbolic Volume and Resolution Proof Width Inequal...

Entry 4e96052b6f7d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 22:15:22 UTC

Minimal Hyperbolic Volume and Resolution Proof Width Inequality

Entry ID: 4e96052b6f7d **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 22:15:22 UTC

1. Conjecture

Field A (mathematical branch): Hyperbolic Geometry **Field B** (complexity object): Resolution Proof Complexity

Statement:

For any CNF formula φ with n variables, the minimal hyperbolic volume ($m_h(\varphi)$) of its associated hyperbolic embedding is linearly correlated with its resolution proof width $w(\varphi)$, such that $m_h(\varphi) = \Theta(w(\varphi))$. Furthermore, for all instances with property P, property Q holds, where P is defined as 'the instance φ has a hyperbolic embedding with minimal volume' and Q is 'the resolution proof width of φ is at least $2 * \log(n)$ '.

Rationale (proposer's reasoning):

Hyperbolic geometry has been used in the analysis of quantum complexity due to its connection with entanglement. The minimal hyperbolic volume could provide a lower bound on the complexity, as it captures the geometric complexity of the problem. This conjecture aims to explore this potential by relating it to resolution proof width, which is a fundamental measure in complexity theory.

Taxonomy category: HyperbolicGeometryResolutionProofComplexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 23bc76de523d1401

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, after running 30 seeds, at least 80% of the CNF formulas with n variables have a resolution proof width at least twice their number of variables ($w(\varphi) \geq 2 * \log(n)$), and the correlation between minimal hyperbolic volume and resolution proof width is significant ($p\text{-value} \leq 0.05$).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    for i in range(m):
        max_row = i
        for j in range(i+1, m):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        for j in range(i+1, m):
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]
    return A

def matrix_multiply(A, B):
    m, n, p = len(A), len(B), len(B[0])
    C = [[0 for _ in range(p)] for _ in range(m)]
    for i in range(m):
        for j in range(p):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def determinant(A):
    m, n = len(A), len(A[0])
    if m != n:
        raise ValueError("Matrix must be square")
    if n == 1:
        return A[0][0]
    det = 0
```

```

for j in range(n):
    submatrix = [row[:j] + row[j+1:] for row in A[1:]]
    det += (-1) ** j * A[0][j] * determinant(submatrix)
return det

def resolution_width(cnf):
    seen = set()
    literals = set(lit for clause in cnf for lit in clause if isinstance(lit, int))
    for literal in literals:
        if literal < 0 and -literal not in seen:
            seen.add(-literal)
    return len(seen)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    cnf = []
    for _ in range(n):
        clause = [random.choice(range(1, n+1)) for _ in range(random.randint(2, n))]
        cnf.append(clause)

    m_h = determinant(gaussian_elimination([[sum(abs(lit) for lit in clause if isinstance(lit, int))
    w = resolution_width(cnf)

    return {
        "metric_name": "resolution_width",
        "metric_value": w,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": m_h == w,
        "counterexample": "" if m_h == w else f"m_h({cnf})={m_h}, w({cnf})={w}"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        for r in results:
            if not r["conjecture_holds"]:
                print(f"RESULT: FALSIFIED counterexample=\"{r['counterexample']}\" first_failing_seed={seed}")
                break

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e52284b3.py", line 87, in <module>
    result = run_trial(seed)
             ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e52284b3.py", line 69, in run_trial
    m_h = determinant(gaussian_elimination([[sum(abs(lit) for lit in clause if isinstance(lit, int)) f
    ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e52284b3.py", line 44, in determinant
    raise ValueError("Matrix must be square")
ValueError: Matrix must be square
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete the required number of seeds to evaluate the conjecture. | next: Re-run the test with a larger number of seeds and ensure that the code does not crash.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15594
2	preregistration	ollama_remote	glm4:latest	0	0	9666
3	novelty	ollama_remote	glm4:latest	0	0	8562
4	novelty	ollama_remote	glm4:latest	0	0	8610
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	34430
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10701
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11013
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10328
9	judge	ollama_remote	glm4:latest	0	0	21063

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 129965 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/4e96052b6f7d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4e96052b6f7d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/4e96052b6f7d.tar.gz` (if generated)